Experiment No 5

Name: Devang Deokule
Roll Number: A-58
TU ID: TU4S2425009

AIM: Implementation of Association Mining (Apriori, FPM) using open source tool WEKA.

Objectives:

After completing this experiment you will be able to:

1. Explain the value of finding associations in market basket data.
2. Characterize the kinds of patterns that can be discovered by association rule mining.
3. Describe how to extend a relational system to find patterns using association rules.

LO's to be achieved:

LON	Lab Outcomes
LO3	Implement the appropriate data mining methods like classification, clustering or association mining on large data sets using open-source tools like WEKA

Theory:

Frequent itemset mining leads to the discovery of associations and correlations among items in large transactional or relational data sets. With massive amounts of data continuously being
collected and stored, many industries are becoming interested in mining such patterns from their databases. The discovery of interesting correlation relationships among huge amounts of business
transaction records can help in many business decision-making processes, such as catalog design, cross-marketing, and customer
shopping behavior analysis.

A typical example of frequent itemset mining is market basket analysis. This process analyzes customer buying habits by finding associations between the different items that customers place in their "shopping baskets". The discovery of such associations can help retailers develop marketing strategies by gaining insight into which items are frequently purchased together by customers.

Frequent Itemsets, Closed Itemsets, and Association Rules

1. An association rule is of the form X => Y,
   Where X = {xl, x2, x3, …, xn} and Y = {y1, y2, y3, …, ym} are sets of items, The Xi and Yj being distinct items for all i and all j. i.e $X \cap Y = \Phi$

2. This association states that if a customer buys X, he or she is also likely to buy Y.
3. In general, any association rule has the form LHS (left-hand side) => RHS (right-hand side), where LHS and RHS are sets of items.
4. The set LHS U RHS is called an itemset, the set of items purchased by customers.
5. For an association rule to be of interest to a data miner, the rule should satisfy some interest measure.
   Two common interest measures are support and confidence.
6. The support for a rule LHS => RHS is with respect to the itemset:
   6.1 It refers to how frequently a specific itemset occurs in the database.
   6.2 That is, the support is the percentage of transactions that contain all of the items in the itemset, LHS U RHS.
   6.3 If the support is low, it implies that there is no overwhelming evidence that items in LHS U RHS occur together, because the itemset occurs in only a small fraction of transactions.
7. The confidence is with regard to the inference shown in the rule.
   7.1 The confidence of the rule LHS => RHS is computed as:

   Support (LHS U RHS) / Support (LHS).

   7.2 We can think of it as the probability that the items in RHS will be purchased given that the items in LHS are purchased by a customer.
   7.3 Another term for confidence is strength of the rule.

8. Example:

TID	Time	Items Bought
101	6.35	Milk, Bread, Cookies, Juice
792	7.38	Milk, Juice
1130	8.05	Milk, Eggs
1735	8.40	Bread, Cookies, Coffee

Figure-2: Example transactions in market-basket model.

   8.1 As an example of support and confidence, consider the following two rules:
   Milk => Juice and Bread => Juice.

   8.2 Looking at our four sample transactions in Figure-2, we see
   that The support of {Milk, Juice} is 50% and
   The support of {Bread. Juice} is only 25%.
   The confidence of Milk => Juice is 66.7% (meaning that, of three transactions in which milk occurs, two contain juice) and
   The confidence of Bread => Juice is 50% (meaning that one of two transactions containing bread also contains juice).
   8.3 As we can see, support and confidence do not necessarily go hand in hand.
   The goal of mining association rules, then, is to generate all possible rules that exceed some minimum user-specified support and confidence thresholds. The problem is thus decomposed into two sub-problems:
   8.3.1 Generate all itemsets that have a support that exceeds the threshold. These sets of items are called large (or frequent) itemsets. Note that large here means large support.

8.3.2 For each large itemset, all the rules that have a minimum confidence are generated as follows:

For a large itemset X and Y $\subset$ X, let Z = X - Y; then if support(X)/support(Z) > minimum confidence, the rule Z => Y (that is, X - Y => Y) is a valid rule.

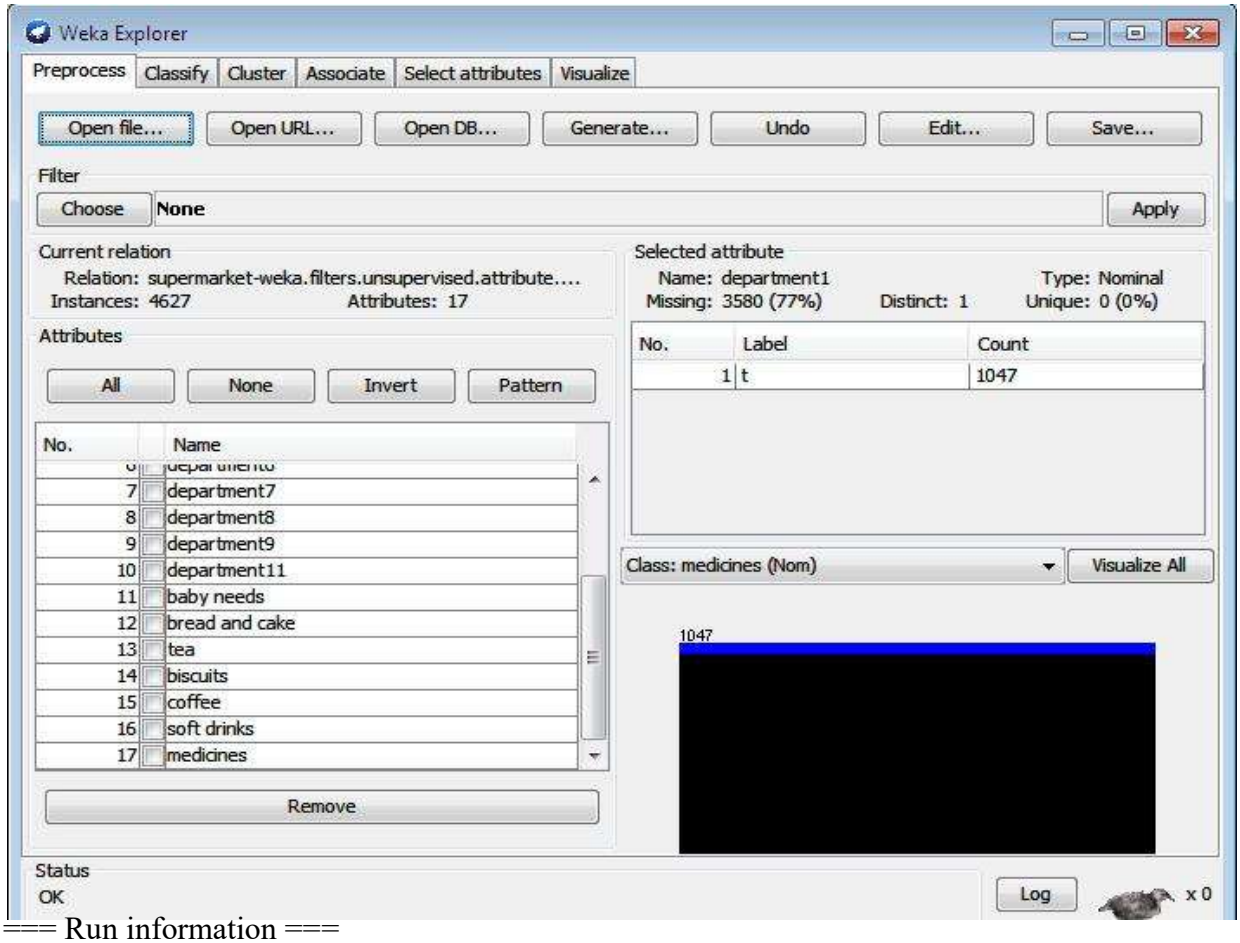
9. Generating rules by using all large itemsets and their supports is relatively straightforward. However, discovering all large itemsets together with the value for their support is a major problem if the cardinality of the set of items is very high. A typical supermarket has thousands of items. The number of distinct itemsets is 2^m, where m is the number of items, and counting support for all possible itemsets becomes very computation-intensive. To reduce the combinatorial search space, algorithms for finding association rules utilize the following properties:
   9.1 A subset of a large itemset must also be large (that is, each subset of a large itemset exceeds the minimum required support). i.e downward closure property.
   9.2 Conversely, a superset of a small itemset is also small (implying that it does not have enough support). i.e antimonotonicity property.

9.3 These two properties help in reducing the search space of possible solutions. That is, once an itemset is found to be small, then any extension to that itemset, formed by adding one or more items to the set, will also yield a small itemset.

<u>Conclusion</u>:

- In the process of finding frequent itemsets, Apriori avoids counting the candidate itemsets that are known to candidates are generated by joining among the frequent also candidate are pruned according the Apriori property.
- As a result the number of remaining candidate itemsets ready for further support checking becomes much smaller, which dramatically reduces the computation, I/O cost and memory requirement.
- The main limitations of Apriori is the number of scan required for generating the frequent itemsset.

Weka Output



=== Run information ===

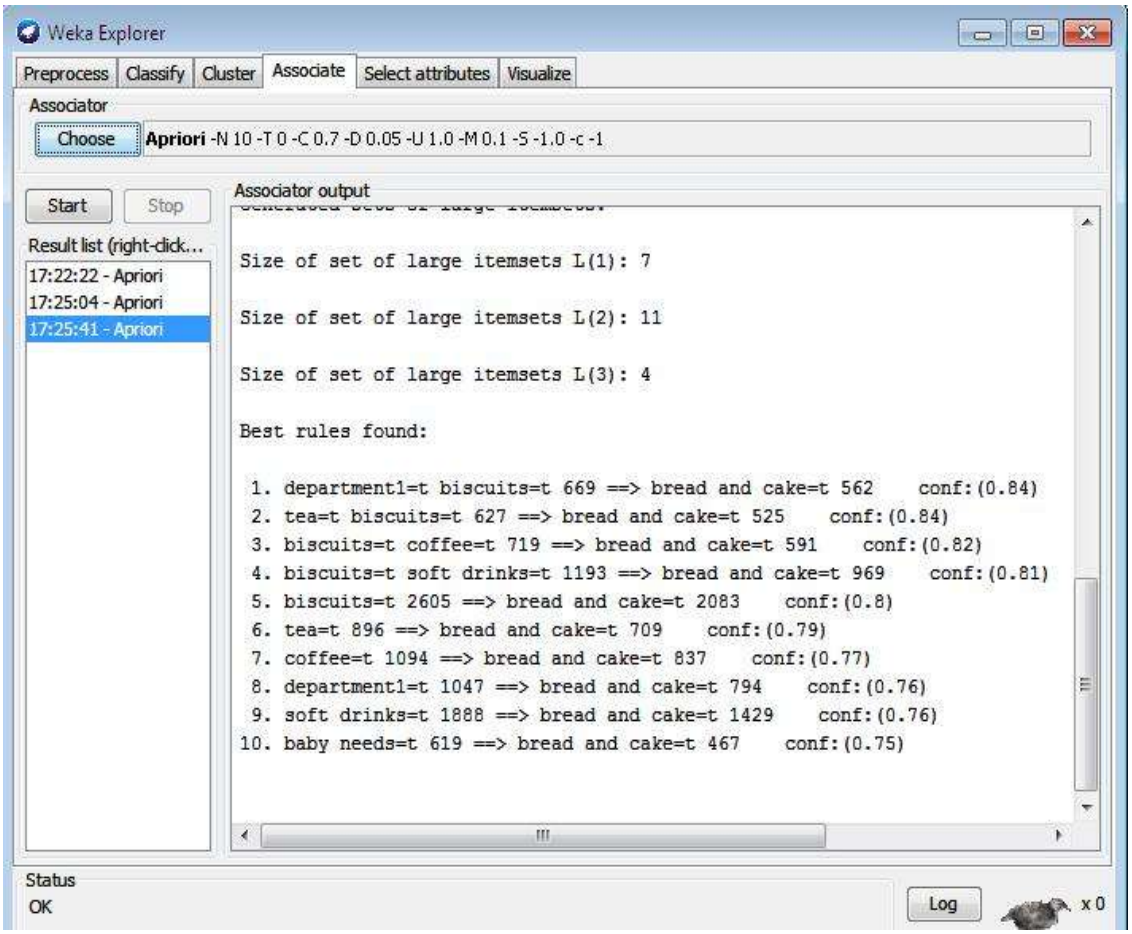
Scheme: weka.associations.Apriori -N 10 -T 0 -C 0.7 -D 0.05 -U 1.0 -M 0.1 -S -1.0 -c -1
Relation : : supermarket-weka.filters.unsupervised.attribute.Remove-R10,14-16,19-25,27-44,46-50,52-217

Instances: 4627 Attributes: 17 department1 department2 department3 department4 department5 department6 department7 department8	Apriori ======= Minimum support: 0.1 (463 instances) Minimum metric <confidence>: 0.7 Number of cycles performed: 18 Generated sets of large itemsets: Size of set of large itemsets L(1): 7 Size of set of large itemsets L(2): 11 Size of set of large itemsets L(3): 4

Best rules found:

1. department1=t biscuits=t 669 ==> bread and cake=t 562 conf:(0.84)

2. tea=t biscuits=t 627 ==> bread and cake=t 525 conf:(0.84)
3. biscuits=t coffee=t 719 ==> bread and cake=t 591 conf:(0.82)
4. biscuits=t soft drinks=t 1193 ==> bread and cake=t 969 conf:(0.81)
5. biscuits=t 2605 ==> bread and cake=t 2083 conf:(0.8)
6. tea=t 896 ==> bread and cake=t 709 conf:(0.79)
7. coffee=t 1094 ==> bread and cake=t 837 conf:(0.77)
8. department1=t 1047 ==> bread and cake=t 794 conf:(0.76)
9. soft drinks=t 1888 ==> bread and cake=t 1429 conf:(0.76)
10. baby needs=t 619 ==> bread and cake=t 467 conf:(0.75)



Post Lab Exercise:

Qu-1: A database has five transactions. Let min sup = 60% and min conf = 80%. find the strong association rules using Apriori algorithm.

TID	Items_bought
T100	{M, O, N, K, E, Y}
T200	{D, O, N, K, E, Y}
T300	{M, A, K, E}
T400	{M, U, C, K, Y}
T500	{C, O, O, K, I, E}